

The Comparison-Sort Lower Bound: Why $O(n \log n)$ Is the Floor

Programming and Data Structures — Week 5, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to model any comparison-based sorting algorithm as a decision tree; explain why such a tree must have at least $n!$ leaves, one for every possible ordering of the input; derive the $\Omega(n \log n)$ lower bound on comparison-sort running time from the height of this tree, using Stirling's approximation to estimate $\log_2(n!)$; and explain precisely why this bound applies only to algorithms that sort by comparison, connecting directly back to Lecture 3's counting sort and radix sort.

Outline

Today covers a 20-questions analogy motivating the decision-tree argument; modeling any comparison-based sorting algorithm as a binary decision tree; why such a tree must have at least $n!$ leaves; deriving the tree's minimum height from its leaf count; applying Stirling's approximation to estimate that height in closed form; assembling the complete proof of the $\Omega(n \log n)$ lower bound; a direct reconnection to Lecture 3's counting sort and radix sort, explaining why they do not violate this result; an in-class quiz; and a summary.

The 20-questions analogy

The children's game "20 questions" illustrates the exact mathematical principle this lecture formalizes: each yes/no question can, at best, narrow down the space of remaining possibilities by half. To reliably identify one specific item out of N possible items, at least $\log_2 N$ questions are required in the worst case — asking fewer guarantees that at least two distinct possibilities remain indistinguishable after all questions are exhausted, meaning the game must fail on at least one of them. Every comparison in a sorting algorithm is exactly this kind of yes/no question, and the "possibilities" being distinguished are the possible orderings of the input.

Modeling a sort as a decision tree

Any comparison-based sorting algorithm can be modeled as a binary decision tree. Each internal node represents one comparison, asking whether $\text{arr}[i] < \text{arr}[j]$ for some pair of positions; the two branches correspond to the “yes” and “no” outcomes. Running the algorithm on a specific input traces exactly one path from the root down to a leaf, following whichever branch each comparison’s actual outcome dictates. Each leaf represents one final, fully determined output ordering the algorithm can produce.

Why the tree needs at least $n!$ leaves

An array of n distinct elements has exactly $n!$ possible orderings, or permutations. A correct sorting algorithm, given every possible input arrangement, must be capable of producing every single one of these $n!$ orderings as its sorted output for the appropriate input. This forces the decision tree to have at least $n!$ distinct leaves — if it had fewer, at least two different required outputs would have to share a single leaf, which is impossible, since a single leaf corresponds to one single, fixed output ordering.

From leaves to height: the height of a binary tree

A binary tree of height h — where every internal node has at most two children, matching the yes/no structure of a comparison — can have at most 2^h leaves, a fact provable by induction on h (doubling the possible leaf count with each additional level). Since the decision tree needs at least $n!$ leaves, this gives $2^h \geq n!$. Solving for h by taking $\log_2$ of both sides: $h \geq \log_2(n!)$. Since the tree’s height represents the worst-case number of comparisons the algorithm might need to make, this directly bounds the algorithm’s worst-case comparison count from below.

Stirling’s approximation, applied

To turn $\log_2(n!)$ into a more familiar closed form, Stirling’s approximation is used: $n! \approx \sqrt{2\pi n} (n/e)^n$. Taking $\log_2$ of both sides and simplifying yields $\log_2(n!) \approx n \log_2 n - n \log_2 e + O(\log n)$. The dominant term, the one that grows fastest as n increases, is $n \log_2 n$; every other term grows more slowly and is absorbed into lower-order notation. This is the same big-O reasoning from Week 1 Lecture 4 — keep the fastest-growing term, discard the rest.

The full proof, assembled

Assembling the complete argument: any comparison-based sorting algorithm corresponds to a binary decision tree (step 1); that tree must have at least $n!$ leaves, since it must be able to produce every possible output ordering (step 2); a binary tree of height h has at most 2^h leaves, forcing $h \geq \log_2(n!)$

(step 3); Stirling's approximation shows $\log_2(n!) = \Theta(n \log n)$ (step 4); therefore, any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case (step 5). This is a genuine mathematical proof, not an empirical observation — no comparison sort, however cleverly designed, can ever beat this bound.

What this means for the algorithms covered so far

All three simple sorting algorithms covered this week — bubble, selection, and insertion sort — are comparison-based, and all three have $O(n^2)$ worst case, which is consistent with, but does not achieve, the $\Omega(n \log n)$ lower bound just proven (since n^2 grows faster than $n \log n$, it does not contradict the bound, it simply falls short of being optimal). None of this week's algorithms actually reach the $\Theta(n \log n)$ bound. Week 6's merge sort and quicksort are the first comparison-based algorithms in this course that do — merge sort guarantees $\Theta(n \log n)$ in every case, while quicksort achieves it on average, with a worst case that this lecture's bound does not prevent from being worse.

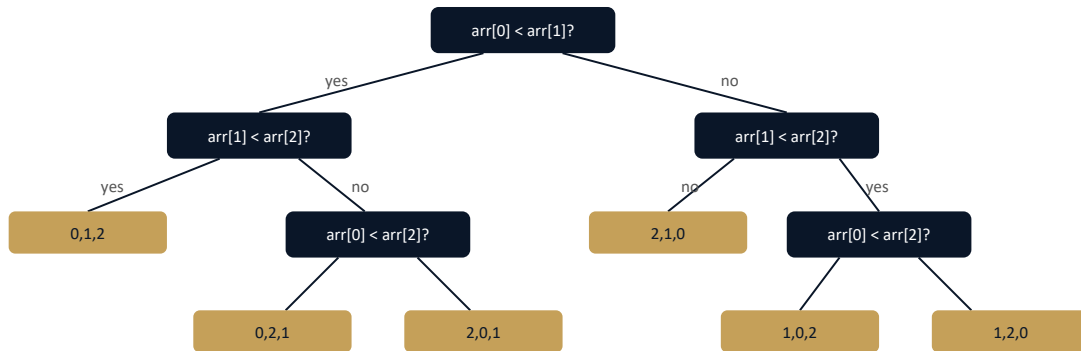
Connecting back: why Lecture 3's algorithms don't violate this

The entire lower-bound proof rests critically on step 1: modeling the algorithm's behavior as a sequence of pairwise comparisons. Counting sort and radix sort, covered in Lecture 3, never ask "is `arr[i]` less than `arr[j]`?" at any point — they extract digit or index values directly and use them to compute array positions arithmetically. Since neither algorithm is built from comparisons in this sense, step 1 of the proof simply does not apply to them, and the $\Omega(n \log n)$ result says nothing whatsoever about their running time. This is precisely why they can achieve $O(n+k)$ or $O(n)$ without any contradiction — they operate in a different computational model than the one the proof concerns.

Working the $n=3$ case by hand, completely

Working through $n = 3$ completely by hand: the first comparison, `arr[0]` versus `arr[1]`, splits into two branches. Each subsequent branch requires one or two further comparisons to fully resolve, and counting all the leaves at the bottom of the tree gives exactly 6, matching $3! = 6$ exactly, with no more and no fewer — confirming both that the tree must have at least this many leaves, and that a well-designed algorithm can achieve exactly this many, with no wasted comparisons.

The picture: a decision tree for $n=3$



6 leaves total = $3!$ — exactly the number of possible orderings, with no waste.

For $n = 3$, there are $3! = 6$ possible orderings, and a correctly functioning comparison sort's decision tree must have at least 6 leaves — this small, fully drawable case makes the abstract argument concrete and checkable by hand.

MTech addition — is the bound tight?

It is worth being precise about the word “optimal” here. The lower bound establishes that $\Omega(n \log n)$ comparisons are necessary in the worst case for any comparison sort. Merge sort, covered in Week 6, achieves exactly $\Theta(n \log n)$ in every case, which means it matches this lower bound asymptotically — no comparison sort can do asymptotically better in the worst case, making merge sort asymptotically optimal. This does not mean no comparison sort can ever be faster on a *specific* input in absolute terms — Lecture 2 already showed insertion sort outperforming any $O(n \log n)$ algorithm in absolute terms on nearly-sorted data, since $O(n)$ beats $O(n \log n)$ for large n . “Optimal” describes worst-case asymptotic behavior, not every possible input.

A numeric check of Stirling's approximation

Checking the claimed relationship numerically for small n : at $n = 5$, $\log_2(5!) = \log_2(120) \approx 6.9$, while $5 \log_2 5 \approx 11.6$ — comparable in order of magnitude, with the gap explained by the lower-order terms Stirling's approximation sets aside. At $n = 10$ and $n = 20$, the two quantities remain proportionally close, confirming $\log_2(n!) = \Theta(n \log n)$ holds in practice, not merely as an asymptotic abstraction disconnected from any concrete number.

Exercise

As an exercise, draw the complete decision tree for sorting 3 elements by hand, using comparisons between the three positions, and confirm it produces exactly 6 distinct leaves, matching $3! = 6$. Separately, compute $\log_2(n!)$ using Stirling's approximation for $n = 10$ and for $n = 20$, and compare these values against $n \log_2 n$ for the same values of n , confirming the two grow at comparable rates, differing only in lower-order terms.

Why this proof style matters beyond sorting

This lecture's proof style is worth recognizing as distinct from everything analyzed so far this course. Every previous complexity result — Big-O in Week 1, amortized analysis in Week 2, the array and linked list trade-offs — analyzed a *specific* algorithm's cost. This lecture instead proves a *lower bound*: a statement about what is impossible for an entire category of algorithms, regardless of how cleverly one might be designed. The decision-tree technique generalizes well beyond sorting: any problem solved by a sequence of binary (yes/no) decisions has an information-theoretic floor determined by how many distinguishable outcomes must be told apart. Recognizing this pattern — “this problem requires distinguishing N possible outcomes, so at least $\Omega(\log N)$ questions are needed” — recurs throughout algorithm design, well beyond this one sorting result.

Quiz — apply the bound

A claimed comparison-based sorting algorithm is said to sort any array of 8 elements using only 10 comparisons in the worst case, regardless of the input. Before checking the answer, determine whether this claim is possible, using this lecture's lower bound.

A second quiz check — $n=4$

As a second check of the same reasoning: could a comparison-based sorting algorithm correctly sort any 4-element array using only 4 comparisons in the worst case, for every possible input? Work this out using the same method as the previous quiz before checking the answer.

Quiz — answer

Sorting 8 elements requires the decision tree to have at least $8! = 40,320$ leaves, one for every possible ordering. A tree using only 10 comparisons in the worst case has height at most 10, giving at most $2^{10} = 1,024$ leaves — far fewer than the 40,320 required. The claim is therefore impossible: no comparison-based algorithm can correctly sort every possible arrangement of 8 elements using only 10 comparisons, since the decision tree simply cannot have enough leaves to distinguish all the required output orderings.

Second quiz — answer

Sorting 4 elements requires the decision tree to have at least $4! = 24$ leaves. A tree of height 4 has at most $2^4 = 16$ leaves, which is fewer than the 24 required — so this claim is also impossible, for exactly the same structural reason as the earlier 8-element case. This confirms the reasoning generalizes correctly rather than being an artifact of the specific numbers chosen in the first quiz.

Summary

Any comparison-based sorting algorithm can be modeled as a binary decision tree, with each leaf corresponding to one possible output ordering. Since this tree must have at least $n!$ leaves, its height is bounded below by $\log_2(n!)$, which Stirling's approximation shows to be $\Theta(n \log n)$. This proves that $\Omega(n \log n)$ comparisons are necessary, in the worst case, for any algorithm that sorts purely by comparison — a genuine mathematical floor, not an empirical tendency. Counting sort and radix sort, covered in Lecture 3, are exempt from this bound entirely, since they do not sort by comparison. This closes Week 5's coverage of simple, comparison-based sorting and its fundamental limit. Next week introduces merge sort and quicksort, the first algorithms in this course that actually reach the $\Theta(n \log n)$ bound just proven.